FINAL PROJECT

202322621

Individual Assignment

Final Report – Moderated Learning Management System

CS4433 Software Engineering · National University of Lesotho · 3 November 2026

Selitse M.S

Abstract

This report summarises the development of a **Moderated Learning Management System** for a secondary school, built as an individual project using the Waterfall model with V-Model verification and validation. The system is an offline C++17 console application with an embedded SQLite database, used on one school computer by administrators, lecturers, Heads of Department (HoDs) and students. Its central rule is that no mark reaches a student until the HoD has moderated and released it, and that every mark change is audited. The project produced a requirements specification (24 functional and 6 non-functional requirements), a four-layer design using the Strategy, Repository and Factory patterns, about 3,500 lines of C++, 51 automated tests with 96% business-logic line coverage, 22 system tests, release v1.0 and, after change request CR1, release v1.1. Following the submission instructions, each section is a summary; the full artefacts, all diagrams (in images/) and the video presentation (in video/) are linked from the GitHub repository.

Contents

1 Abstract and contents 2
2 Introduction 2
3 Project management plan 2
4 Requirements engineering 2
5 Software design 3
6 Implementation 3
7 Verification and validation 3
8 Configuration management and QA 3
9 Deployment 3
10 Maintenance (CR1) 4
11 Professional, legal, ethical issues 4
12 Conclusion 4
13 References 4
14 Appendices 4

2. Introduction

Problem identified. The school assesses work on paper, and its stakeholders disagree: lecturers want students to get marks as soon as work is marked, while the Principal insists that no mark is seen before the HoD has checked it. Late work is treated inconsistently, there is no record of who changed a mark, when or why, the computers are old, the internet is unreliable, and teachers are not IT experts. **Aim:** a simple, offline system that enforces moderation, applies one late-penalty rule to everyone and records every mark change. Payments, video, automated grading, plagiarism detection and web or mobile interfaces are out of scope.

3. Project management plan

The project management plan assessed technical, operational, economic, legal and schedule feasibility and recommended proceeding: C++17 and SQLite meet the offline requirement with free tools, while schedule was the largest risk. A work breakdown structure and Gantt chart set three phase gates and the critical path 1.0 → 2.0 → 3.0 → 4.0 → 5.0 → 7.0 → 8.0 → 9.0, with CI (6.0) run in parallel with implementation. The risk register scored eight risks by likelihood × impact; only R1 (requirements and design overrunning into implementation) was High. The updated risk register closes R1–R5 and R7, keeps R6 and R8 open until submission and the viva, and adds seven risks that appeared during the project, for example the Windows tool-chain failing to download (R9), the change request breaking existing behaviour (R11) and the report limit changing to five pages (R12).

4. Requirements engineering

The stakeholder analysis, SRS and use case model classified the five stakeholders by viewpoint and resolved four conflicts in writing: **C1** the Principal's rule is adopted (marks hidden until HoD release) but written feedback is shown as soon as work is marked, which meets part of the lecturer's need; **C2** the HoD's penalty (10% of the maximum mark per day for up to three days, then zero) is combined with the student representatives' demand that late work is accepted, not refused; **C3** class-wide extensions were left out of v1.0 and later arrived as CR1; **C4** "simple" was restated as a measurable usability requirement. The SRS follows ISO/IEC/IEEE 29148, with FR01–FR23 (FR24 added in v1.1) and NFR01–NFR06, each prioritised with MoSCoW and given a verification method. The use case diagram covers all four actors with include, extend and generalisation, and three use cases, including *Submit assignment*, are fully dressed. The two main requirements of each kind are:

ID	Requirement (summarised)	Use case and evidence
FR13	A student cannot see any mark until the HoD has moderated and released it (C1)	Moderate marks, Release marks; IT-12, ST-18
FR16	A student can submit up to the (extended) deadline; late work is accepted and flagged	Submit assignment; IT-07, IT-08, ST-10, ST-13
NFR01	Login completes in under 2 s with 800 students stored	Log in; AT-01 measured about 80 ms
NFR03	All core functions work with no internet connection	All use cases; AT-03: 20/20 system tests offline

5. Software design

The software design document uses a **strict four-layer architecture** (presentation, business logic, data access, domain), shown in the component diagram. Each layer uses only the one below it, through interfaces, so the business logic can be tested with a real but temporary database, there is no server (NFR03), and every permission check sits in one layer. **Rejected alternatives:** MVC, because a console shows one screen at a time and MVC does not say where database code belongs; a client–server database, because it needs a network; and relaxed layering, because a menu reading tables directly could bypass the rule that marks stay hidden. The proposed solution in class form is shown in three diagrams: the domain classes (an abstract User with four role subclasses, Course, Assignment, Submission, Extension, AuditEntry), the business-logic services and the data-access layer. Three patterns are applied: **Strategy** (ILatePenaltyPolicy, so the disputed penalty rule can change without editing MarkingService), **Repository** (all SQL behind five interfaces) and **Factory** (UserFactory, MenuFactory). Observer was considered for audit logging and rejected, because an audit entry must be written in the same transaction as the mark change. The design follows SOLID, notably single responsibility per service and dependency inversion through constructor injection. Behaviour is modelled in a submission state machine (SUBMITTED/LATE → MARKED → MODERATED → RELEASED), a swim-lane activity diagram and sequence diagrams for submitting and moderating. The normalised database mirrors the class diagram, with CHECK constraints, an append-only audit table protected by triggers, and a view that hides unreleased marks.

6. Implementation

The source code has one folder and one CMake library per layer, and follows the documented coding standard. Key decisions: services receive a Session and call Session::require() before any action (FR21) and check ownership (own course, own department, own results); marking and its audit entry are written in **one transaction** through IUnitOfWork, so a mark is never changed without a record (FR22); every SQL statement uses ? parameters; passwords are stored only as salted **Argon2id** hashes from libsodium, a recognised, memory-hard algorithm, and are hidden while typed (NFR04); submitted files are copied into the data folder so a student's USB drive can be removed; and all errors are typed exceptions caught by the menu loop, with input validated so that letters, empty input or bad dates never crash the program. The build guide is in the README.

7. Verification and validation

The test plan and results map each V-Model level to the artefact it verifies, with entry and exit criteria, and use equivalence partitioning and boundary values for deadlines, late days (e.g. exactly at the deadline, +1 s, +72 h, +72 h 1 s) and the mark range. Automated tests use Catch2 and run in CI on every push. Two faults were planted on purpose to confirm that the tests detect them.

Level (verifies)	Tests (v1.1)	Result
Unit (detailed design)	UT-01 to UT-25	25 / 25 pass
Integration (architecture)	IT-01 to IT-26, real SQLite database	26 / 26 pass; with unit tests, 349 checks; 96% line coverage of the business-logic layer (target 70%)
System (use cases)	ST-01 to ST-22, script drives the real program, incl. exception flows	22 / 22 pass; transcript kept for each test
Acceptance (NFRs)	AT-01 to AT-06	5 pass with measurements; AT-02 (usability) needs a participant

The defect log records 11 defects (DEF-01 to DEF-11), with severity, cause, fix and retest: for example DEF-08 (High), where only the path of a submitted file was saved, fixed by copying the file into storage. None is open. Every requirement is traced to its use case, design elements, source files and tests in the traceability matrix.

8. Configuration management and quality assurance

Configuration management uses Git with feature branches merged by pull request, commit messages that name the requirement or defect, and annotated tags v1.0 and v1.1 with semantic versioning. A GitHub Actions pipeline runs cppcheck, builds with warnings as errors, runs all automated and system tests, and fails if coverage drops below 70%. For quality assurance, cppcheck reported 29 findings and clang-tidy 14; all were fixed or justified. Metrics: 3,539 lines in 66 files, 294 functions, average cyclomatic complexity 2.0. The self code review against a checklist led to six changes, including splitting three menu functions whose complexity was above 15 (e.g. AdminMenu::handle from 27 to 16).

9. Deployment

The deployment diagram shows one Windows computer running lms.exe with its DLLs and a local data folder, and a USB drive for submissions and backups. An installation guide written for the school technician covers installation, the

first administrator, data protection settings, weekly backup and upgrading. There is a user manual for each role and release notes. Release v1.0 was tagged and published on the GitHub Releases page as a ready-to-run zip.

10. Maintenance (change request CR1)

CR1 asks for class-wide extensions shown on the student's assignment list. The change analysis classifies it as **perfective** maintenance (new functionality requested after release; v1.0 met its requirements) and lists every affected requirement (new FR24; FR15 and FR16 changed), diagram, class, table and test. A class extension must be granted before the original deadline, so no submission already marked LATE needs to be re-classified and the state machine is unchanged. v1.1 added a class_extension table, which a v1.0 database receives automatically, and the updated diagrams show the changes in blue. Five automated and two system tests were added, and the whole v1.0 suite was re-run as a regression suite: all 51 automated and 22 system tests pass.

11. Professional, legal and ethical issues

This section covers four issues. **Data protection:** under Lesotho's Data Protection Act 2011, learners' records are sensitive when they concern children, so the system stores only names, usernames and password hashes, works offline, and restricts each role to its own data; parental consent, a retention period and a breach procedure are recommended to the school. **Security of records:** Argon2id hashes, release-only visibility, an append-only audit log and parameterised SQL; the unencrypted database file is mitigated by folder permissions and BitLocker. **Accessibility:** the keyboard-only, text-based console works with screen readers, in line with the Persons with Disability Equity Act 2021. **Fairness:** one penalty rule applies to everyone, with extensions and recorded HoD adjustments; rounding a few seconds up to a whole day is a known harshness.

12. Conclusion

All 24 functional requirements are implemented and tested, five of the six non-functional requirements are met with measurements, and the system has been released twice. **Challenges:** installing the Windows tool-chain over an unreliable connection, GitHub credential and token problems, and keeping the diagrams consistent after CR1. **Gaps:** usability test AT-02 still needs an untrained participant; only one user can work at a time; the database is not encrypted; there is no deletion of old records; and lecturers cannot list the students on their course. These are the first candidates for a future version.

13. References

ACM/IEEE-CS Joint Task Force (1999) *Software Engineering Code of Ethics and Professional Practice*. New York: ACM.

Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994) *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley.

ISO/IEC/IEEE (2018) *ISO/IEC/IEEE 29148:2018 Systems and software engineering – Life cycle processes – Requirements engineering*. Geneva: ISO.

ISO/IEC/IEEE (2022) *ISO/IEC/IEEE 14764:2022 Software engineering – Software life cycle processes – Maintenance*. Geneva: ISO.

Kingdom of Lesotho (2012) *Data Protection Act, 2011* (Act No. 5 of 2012). Available at: https://lesotholii.org/akn/ls/act/2012/5.

Kingdom of Lesotho (2021) *Persons with Disability Equity Act, 2021* (Act No. 2 of 2021). Available at: https://lesotholii.org/akn/ls/act/2021/2.

libsodium (2025) *Password hashing (Argon2)*. Available at: https://doc.libsodium.org/.

Martin, R.C. (2017) *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Boston: Prentice Hall.

Sommerville, I. (2016) *Software Engineering*. 10th edn. Harlow: Pearson.

14. Appendices

Kept as separate files in the repository: **A** traceability matrix; **B** full test results and CI runs; **C** user manuals; **D** updated risk register; **E** all diagrams, v1.0 and v1.1; **F** video presentation.